



Politechnika
Wrocławska

Struktury danych	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>poniedziałek TN 11¹⁵ – 13⁰⁰</i>
Imię, nazwisko, numer albumu <i>Jan Hulbój 284197, Maciej Polanowski 284195</i>	Data <i>13.04.2026</i>
Link do projektu https://github.com/mlody-jano/dataStructures	

SPRAWOZDANIE – MINIPROJEKT NR 1

Poniższy dokument jest kompleksowym sprawozdaniem opisującym badania przeprowadzone w ramach pierwszego projektu na przedmiocie **Struktury Danych**. Obiektem pierwszego projektu była samodzielna implementacja 3 wybranych struktur danych, które są rzeczywistym odzwierciedleniem wybranego **ADT** - w tym przypadku **Listy**. Każda ze struktur została zbadana pod kątem złożoności obliczeniowej wybranych operacji, które z reguły na niej się wykonuje. W tym przypadku badanymi operacjami było: **dodawanie elementu**, **usuwanie elementu** oraz **wyszukiwanie elementu**.

Spis treści

1 Wstęp teoretyczny	2
1.1 Lista jako ADT	2
1.2 Tablica dynamiczna	2
1.3 Lista wiązana	3
1.4 Lista dwukierunkowa	4
2 Założenia projektowe	5
3 Wyniki badań	7
3.1 Operacje dodawania	7
3.2 Operacje usuwania	8
3.3 Operacje wyszukiwania	9
4 Wnioski	10

Spis rysunków

1	Tablica dynamiczna.	3
2	Lista wiązana.	4
3	Lista dwukierunkowa.	5
4	Skrypt generujący dane.	6
5	Fragment kodu odpowiedzialny za pomiar czasu.	7
6	Złożoności obliczeniowe operacji dodawania.	8

7	Złożoności obliczeniowe operacji usuwania.	9
8	Złożoności obliczeniowe operacji wyszukiwania.	9

Spis tabel

1	Oczekiwana złożoność obliczeniowa wybranych operacji dla tablicy dynamicznej	3
2	Oczekiwana złożoność obliczeniowa wybranych operacji dla listy wiązanej	4
3	Oczekiwana złożoność obliczeniowa wybranych operacji dla listy dwukierunkowej	5
4	Wyniki uzyskane dla operacji dodawania	7
5	Wyniki uzyskane dla operacji usuwania	8
6	Wyniki uzyskane dla operacji wyszukiwania	9

1 Wstęp teoretyczny

1.1 Lista jako ADT

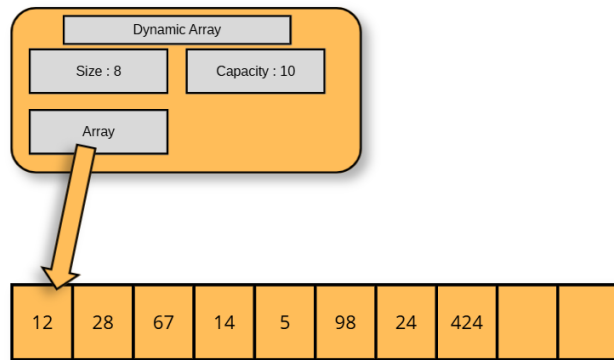
Abstrakcyjny typ danych tj. **ADT** jest tworem ideowym, który pozwala zdefiniować interfejs oraz własności struktury tworzonej, pozostawiając definicję poszczególnych operacji, w gestii odpowiedniej implementacji. Taki proces pozwala stworzyć pewnego rodzaju polimorficzne podejście, w którym wybrane ADT stanowi fundament do tworzenia jego implementacji, w różne, nie zawsze takie same sposoby.

1.2 Tablica dynamiczna

Tablica dynamiczna (*Dynamic array*) jest to pierwsza z implementacji ADT Listy, która opiera się na tablicy wartości zapisywanej w pamięci. Kluczowymi cechami tablicy dynamicznej jest możliwość zmiany jej rozmiaru podczas działania programu, oraz alokacja w pamięci w postaci ciągłego obszaru danych. Pierwsza z tych cech zapewnia twórcom oprogramowania, że nie musi on z góry znać wielkości danych wejściowych, aby zapewnić odpowiednią ilość miejsca do ich przechowania, gdyż w przypadku wypełnienia tablicy, zamoistnie zmieni ona rozmiar, tak aby zapewnić nowe miejsce w kolekcji dla kolejnego elementu. Z kolei druga cecha tablicy dynamicznej jest równie ważna, ponieważ zapewnia stały dostęp do każdego elementu z kolekcji, ze względu na ciągłość obszaru alokowanego. Odbywa się to przy wykorzystaniu arytmetyki wskaźników, które pozwalają w stałym czasie uzyskiwać dostęp do danych. **W tym projekcie tablica dynamiczna została wykonana w sposób z amortyzowaną zmianą rozmiaru, tj. w przypadku gdy do tablicy wpisywany jest ostatni element, rozmiar tablicy jest podwajany, tak aby uniknąć nadmiernego tworzenia nowych tablic, oraz przepisywania danych.**

Rzeczywiste zastosowania:

- Buforowanie multimediiów: Wykorzystywana w silnikach gier i dekodernach wideo do przechowywania aktualnej klatki obrazu lub paczek audio, gdzie wymagany jest natychmiastowy dostęp w czasie stałym $O(1)$ do konkretnego piksela.
- Komunikacja z GPU: Karty graficzne z reguły wymagają danych przesłanych jako jeden, ciągły blok pamięci (np. współrzędne wierzchołków brył trójwymiarowych). Użycie tutaj list powiązanych jest niemożliwe.



Rysunek 1: Tablica dynamiczna.

Oczekiwane złożoności obliczeniowe operacji na tablicy dynamicznej (z użyciem amortyzacji podwojenia rozmiaru!) wyglądają następująco:

Tabela 1: Oczekiwana złożoność obliczeniowa wybranych operacji dla **tablicy dynamicznej**

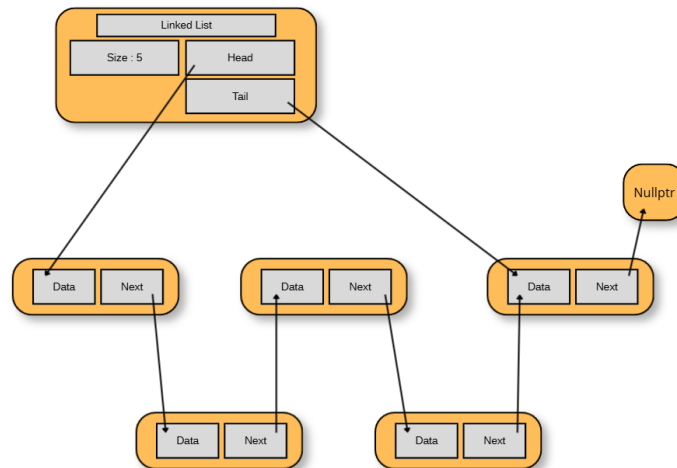
Lp.	Operacja	Optymistycznie	Średnio	Pesymistycznie
1.	pushFront(x)	$O(n)$	$O(n)$	$O(n)$
2.	pushBack(x)	$O(1)$	$O(1)$	$O(1)$
3.	insert(x, i)	$O(1)$	$O(n)$	$O(n)$
4.	popFront(x)	$O(n)$	$O(n)$	$O(n)$
5.	popBack(x)	$O(1)$	$O(1)$	$O(1)$
6.	extract(i)	$O(1)$	$O(n)$	$O(n)$
7.	search(x)	$O(1)$	$O(n)$	$O(n)$

1.3 Lista wiązana

Lista wiązana (*Linked List*) jest kolejną strukturą danych opartą na ADT Listy, która opiera się na połączonych ze sobą **węzłach**, stanowiących kolekcję. Każdy węzeł posiada swoją wartość, oraz wskaźnik na kolejny węzeł, który zapewnia dostęp do następujących elementów. Taka implementacja gwarantuje możliwość przechowywania elementów nie stanowiących ciągłego bloku pamięci, w połączoną (dla programu) kolekcję. Jedną z implementacji listy wiązanej jest wariant z wykorzystaniem dwóch wskaźników, **head** oraz **tail**, które gwarantują możliwość dostępu do każdego z końców listy w czasie $O(1)$, co pozwala na szybką zmianę rozmiaru struktury.

Rzeczywiste zastosowania:

- Technika łańcuchowania (Chaining) w Hash Tables: Powszechnie wykorzystywana w implementacji słowników (map) do radzenia sobie z kolizjami. Elementy o tym samym kluczu hashującym są dopinane jako kolejne węzły do listy jednokierunkowej, nie wymuszając kosztownej przebudowy głównej tablicy.
- Kolejowanie zdarzeń i asynchroniczność: Idealna do tworzenia prostych kolejek typu FIFO o nieograniczonej wielkości, powszechnie stosowanych w mikroservisach do odbierania napływających zadań (np. przychodzące pakiety sieciowe).



Rysunek 2: Lista wiązana.

Oczekiwane złożoności obliczeniowe operacji na liście wiązanej wyglądają następująco:

Tabela 2: Oczekiwana złożoność obliczeniowa wybranych operacji dla **listy wiązanej**

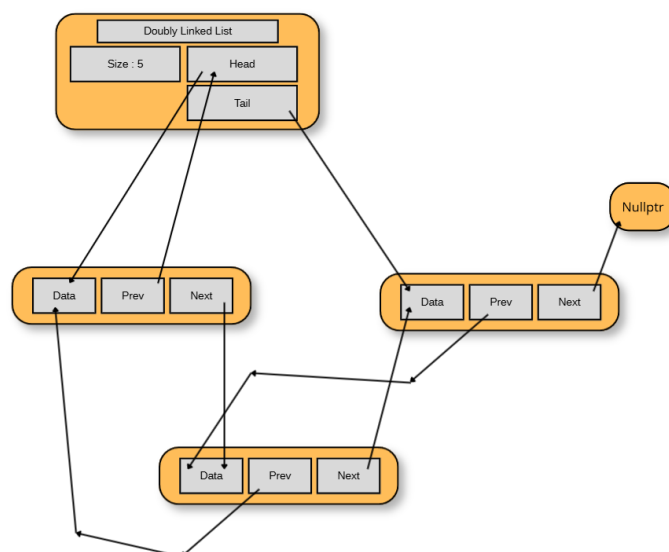
Lp.	Operacja	Optymistycznie	Średnio	Pesymistycznie
1.	pushFront(x)	$O(1)$	$O(1)$	$O(1)$
2.	pushBack(x)	$O(1)$	$O(1)$	$O(1)$
3.	insert(x, i)	$O(1)$	$O(n)$	$O(n)$
4.	popFront(x)	$O(1)$	$O(1)$	$O(1)$
5.	popBack(x)	$O(n)$	$O(n)$	$O(n)$
6.	extract(i)	$O(1)$	$O(n)$	$O(n)$
7.	search(x)	$O(1)$	$O(n)$	$O(n)$

1.4 Lista dwukierunkowa

Lista dwukierunkowa (Doubly Linked List) jest rozszerzeniem struktury listy wiązanej, ponieważ rozszerza budowę pojedynczego węzła o wskaźnik na poprzedni element, pozwalając na swobodne przechodzenie w przód i w tył listy. Umożliwia to usuwanie elementów z końca listy w stałym czasie $O(1)$, a także może zmniejszyć czas wyszukiwania elementów na wybranych pozycjach.

Rzeczywiste zastosowania:

- Pamięć podręczna LRU (Least Recently Used Cache): W połączeniu z tablicą mieszającą służy do buforowania najczęstszych zapytań do baz danych. Kiedy pojawia się nowe zapytanie, użyty węzeł można w czasie stałym wypiąć z dowolnego miejsca listy i przepiąć na jej początek. Gdy zabraknie miejsca, usuwany jest ostatni element (w czasie stałym $O(1)$).
- Nawigacja w aplikacjach (Undo/Redo, Wstecz/Dalej): Architektura listy podwójnie wiązanej stanowi naturalny wzorzec do implementacji przeglądarkowej historii, odtwarzaczy muzycznych (poprzedni utwór/następny) oraz operacji cofania akcji w edytorach tekstu (np. MS Word), ułatwiając płynne poruszanie się w dwóch kierunkach po wektorze czasu lub stronach.



Rysunek 3: Lista dwukierunkowa.

Oczekiwane złożoności obliczeniowe operacji na liście związanej wyglądają następująco:

Tabela 3: Oczekiwana złożoność obliczeniowa wybranych operacji dla **listy dwukierunkowej**

Lp.	Operacja	Optymistycznie	Średnio	Pesymistycznie
1.	pushFront(x)	$O(1)$	$O(1)$	$O(1)$
2.	pushBack(x)	$O(1)$	$O(1)$	$O(1)$
3.	insert(x, i)	$O(1)$	$O(n)$	$O(n)$
4.	popFront(x)	$O(1)$	$O(1)$	$O(1)$
5.	popBack(x)	$O(1)$	$O(1)$	$O(1)$
6.	extract(i)	$O(1)$	$O(n)$	$O(n)$
7.	search(x)	$O(1)$	$O(n)$	$O(n)$

2 Założenia projektowe

W celu zapewnienia odpowiedniego podłoża merytorycznego do wyciągnięcia wniosków dotyczących badanych struktur danych, przyjęte zostały następujące założenia:

- **Rozmiary struktur:** 8 rozmiarów z zakresu 20000 - 160000, skok co 20 tys.
- **Ilość powtórzeń operacji:** 100
- **Sposób generacji danych:** Skrypt Python generujący dane do pliku `.txt`.
- **Sposób pomiaru czasu:** Uśrednienie czasu operacji dla wybranej liczby kopii.

Testy utworzonych struktur danych były prowadzone na urządzeniu wyposażonym w następujące komponenty:

- **Procesor** - Apple M4 10 Core CPU 4Ghz
- **RAM** - 16GB

Dane testowe zostały wygenerowane na podstawie następującego skryptu wykonanego w języku Python. Zastosowanie wybranej wartości ziarna podczas generacji pozwala na uzyskanie odtwarzalnych wyników. Poniżej zawarty został fragment kodu odpowiedzialny za generację danych:

```

1  import random
2  import sys
3
4  def generate_test_data(filename, count, seed_value = 0):
5
6      random.seed(seed_value)                                # Setting the seed for reproducibility
7
8      min_val = -2147483648                                   # Minimum value for 4 byte signed integer
9      max_val = 2147483647                                    # Maximum value for 4 byte signed integer
10
11     print(f"Generowanie {count} rekordów do pliku '{filename}'...")
12     print(f"Użyte ziarno (seed): {seed_value}")
13
14     try:
15         with open(filename, 'w') as f:
16             for i in range(count):
17                 f.write(f"{random.randint(min_val, max_val)}\n")    # Generating random integers from specified range and saving to file
18                 print("Plik wygenerowany pomyślnie!")
19     except IOError as e:
20         print(f"Błąd zapisu do pliku: {e}")
21
22 if __name__ == "__main__":
23     SEED = int(sys.argv[1])                                       # Catching seed value from command line arguments
24     FILE_NAME = f"testing/data/data_{SEED}.txt"                 # Specyfing data file
25     RECORD_COUNT = 2000000                                       # Specyfing number of records to generate
26
27     generate_test_data(FILE_NAME, RECORD_COUNT, SEED)

```

Rysunek 4: Skrypt generujący dane.

Badanie czasu operacji zostało zrealizowane w postaci uśrednienia wykonania operacji na **100** kopiach danej struktury, w celu zapewnienia tych samych warunków początkowych każdej operacji. Fragment kodu odpowiedzialnego za pomiar czasu:

```
auto start = high_resolution_clock::now();

for (int i = 0; i < NUM_COPIES; ++i)
{
    op(containers[i]);
}

auto end = high_resolution_clock::now();

auto totalDuration = duration_cast<nanoseconds>(end - start).count();
long long avgTime = totalDuration / NUM_COPIES;
```

Rysunek 5: Fragment kodu odpowiedzialny za pomiar czasu.

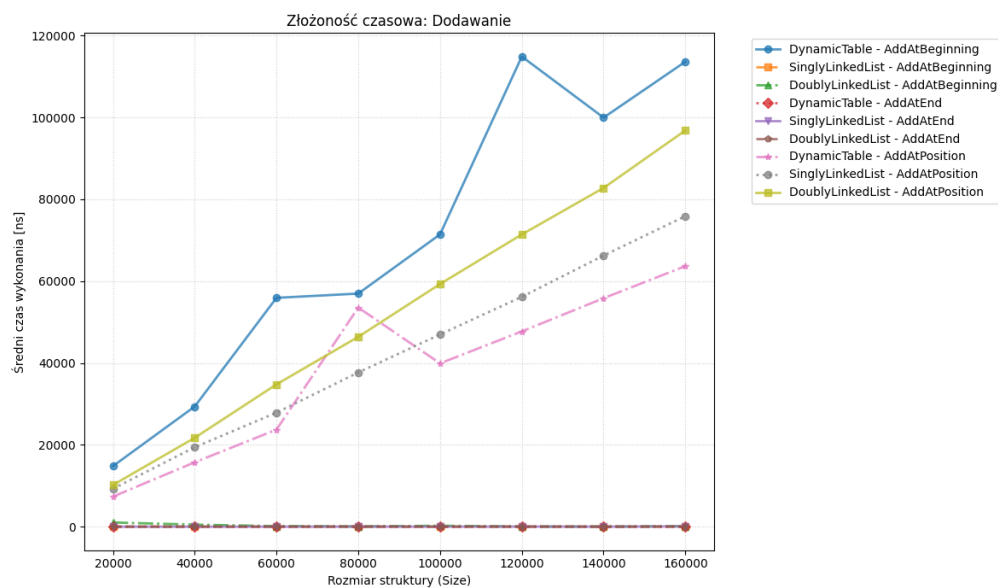
3 Wyniki badań

3.1 Operacje dodawania

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji **dodawania**, zarówno dla różnych miejsc w strukturze, jak i dla różnych struktur. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 4: Wyniki uzyskane dla operacji **dodawania**.

Lp.	Data Structure	Operation	n = 20k	n = 40k	n = 60k	n = 80k	n = 100k	n = 120k	n = 140k	n = 160k
1.	DynamicTable	addAtBeggining()	15182ns	28954ns	43919ns	58314ns	71333ns	92377ns	99642ns	114938ns
2.	SinglyLinkedList	addAtBeggining()	21ns	40ns	22ns	19ns	15ns	17ns	10ns	17ns
3.	DoublyLinkedList	addAtBeggining()	188ns	85ns	67ns	49ns	44ns	46ns	40ns	48ns
4.	DynamicTable	addAtEnd()	4ns	6ns	5ns	5ns	5ns	7ns	10ns	18ns
5.	SinglyLinkedList	addAtEnd()	13ns	13ns	36ns	22ns	21ns	14ns	28ns	80ns
6.	DoublyLinkedList	addAtEnd()	13ns	15ns	27ns	32ns	16ns	22ns	73ns	41ns
7.	DynamicTable	addAtPosition()	7428ns	15577ns	23465ns	31977ns	39725ns	48977ns	55700ns	63612ns
8.	SinglyLinkedList	addAtPosition()	9177ns	18633ns	28170ns	37692ns	47480ns	57007ns	65998ns	75200ns
9.	DoublyLinkedList	addAtPosition()	9949ns	22354ns	33822ns	45842ns	58870ns	71982ns	82223ns	96750ns



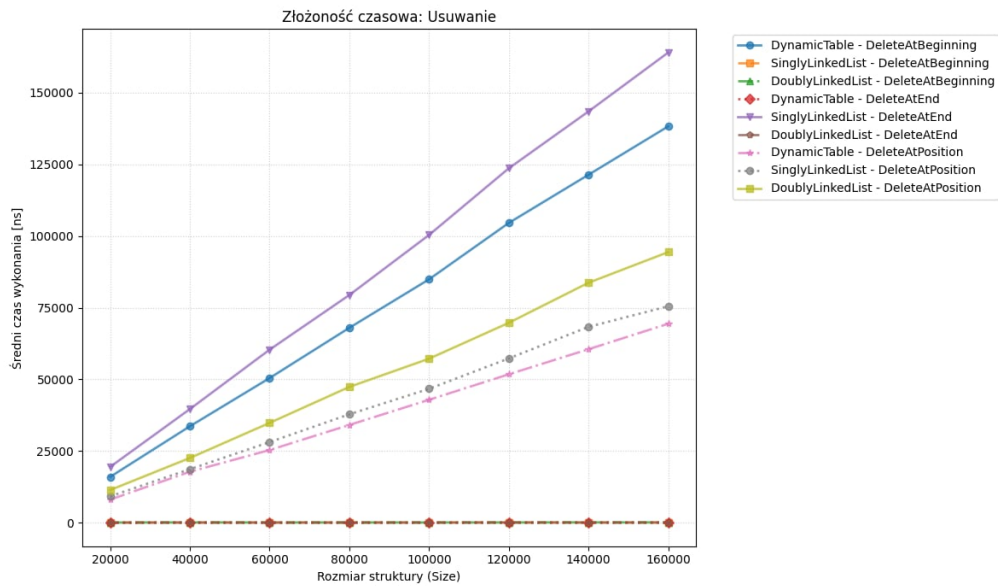
Rysunek 6: Złożoności obliczeniowe operacji dodawania.

3.2 Operacje usuwania

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji **usuwania**, zarówno dla różnych miejsc w strukturze, jak i dla różnych struktur. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 5: Wyniki uzyskane dla operacji **usuwania**.

Lp.	Data Structure	Operation	n = 20k	n = 40k	n = 60k	n = 80k	n = 100k	n = 120k	n = 140k	n = 160k
1.	DynamicTable	deleteAtBeginning()	16092ns	34156ns	51350ns	70056ns	85949ns	105162ns	122012ns	140682ns
2.	SinglyLinkedList	deleteAtBeginning()	37ns	42ns	54ns	50ns	52ns	58ns	80ns	120ns
3.	DoublyLinkedList	deleteAtBeginning()	74ns	366ns	75ns	93ns	108ns	88ns	97ns	87ns
4.	DynamicTable	deleteAtEnd()	3ns	130ns	3ns	5ns	7ns	8ns	6ns	18ns
5.	SinglyLinkedList	deleteAtEnd()	19539ns	39650ns	60232ns	82140ns	101286ns	124596ns	147846ns	167672ns
6.	DoublyLinkedList	deleteAtEnd()	62ns	70ns	80ns	81ns	82ns	110ns	76ns	87ns
7.	DynamicTable	deleteAtPosition()	8362ns	16692ns	26094ns	36034ns	43497ns	52572ns	61215ns	70125ns
8.	SinglyLinkedList	deleteAtPosition()	9565ns	18968ns	28975ns	38071ns	48607ns	56635ns	66670ns	75413ns
9.	DoublyLinkedList	deleteAtPosition()	11588ns	22479ns	34436ns	49403ns	61257ns	72788ns	82143ns	93896ns



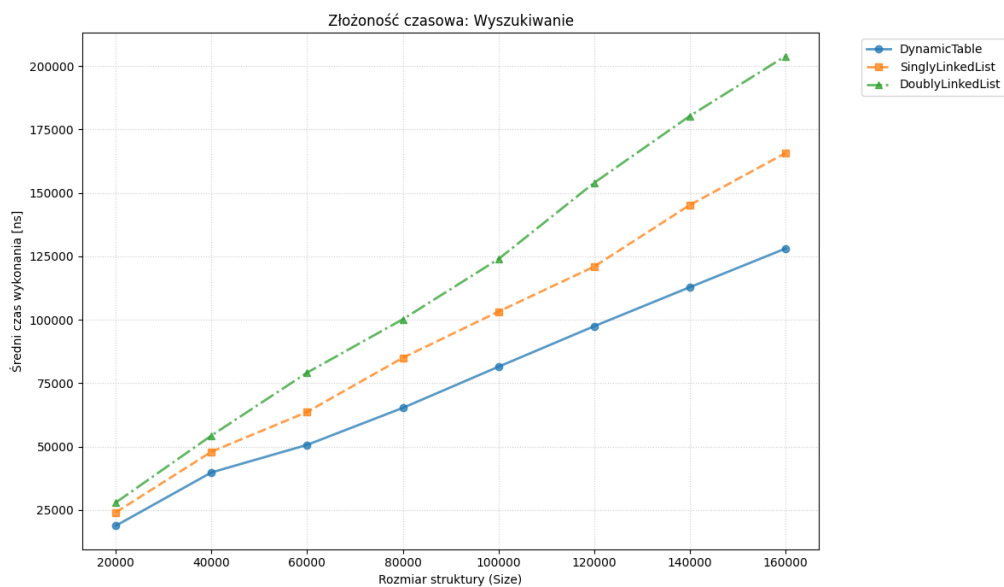
Rysunek 7: Złożoności obliczeniowe operacji usuwania.

3.3 Operacje wyszukiwania

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji **wyszukiwania**, zarówno dla różnych miejsc w strukturze, jak i dla różnych struktur. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 6: Wyniki uzyskane dla operacji **wyszukiwania**.

Lp.	Data Structure	Operation	n = 20k	n = 40k	n = 60k	n = 80k	n = 100k	n = 120k	n = 140k	n = 160k
1.	DynamicTable	search()	18417ns	35418ns	50437ns	65482ns	82527ns	97416ns	113921ns	129731ns
2.	SinglyLinkedList	search()	23252ns	44627ns	64665ns	83655ns	101887ns	121340ns	143123ns	163231ns
3.	DoublyLinkedList	search()	28810ns	51961ns	75301ns	99837ns	119865ns	148315ns	171195ns	192527ns



Rysunek 8: Złożoności obliczeniowe operacji wyszukiwania.

4 Wnioski

Przeprowadzone badania empiryczne potwierdzają założenia teoretyczne dotyczące złożoności obliczeniowej abstrakcyjnych struktur danych. Osiągnięte wyniki pozwalają na sformułowanie następujących konkluzji:

Dobór struktury determinuje wydajność: Wykresy wyraźnie obrazują, że nie istnieje struktura idealna. Tablica dynamiczna deklasuje liste wiązań jednokierunkową w przypadku usuwania na końcu kolekcji (uzyskując stały czas $O(1)$). Jest jednak wysoce nieefektywna przy modyfikacjach na początku struktury ($O(n)$), gdzie z kolei dominują listy powiązane, umożliwiając przepinanie wskaźników w czasie stałym.

Znaczenie pamięci ciągłej: Pomimo że teoretycznie operacja iteracji po wszystkich węzłach ma zawsze złożoność $O(n)$, tablice dynamiczne w realnych scenariuszach powinny zyskiwać przewagę podczas wyszukiwania i przechodzenia po elementach dzięki zjawisku lokalności pamięci podręcznej (ang. *cache locality*). Narzut na rozwiązywanie referencji wskaźnikowych w listach powiązanych stwarza wąskie gardło przy dużych wolumenach danych.

Lista dwukierunkowa vs jednokierunkowa: Rozszerzenie pojedynczego węzła o wskaźnik na element poprzedni w liście dwukierunkowej dramatycznie redukuje czas potrzebny na usunięcie z jej końca z $O(n)$ do $O(1)$. Ta optymalizacja odbywa się jednak kosztem większego zużycia pamięci na przechowywanie dodatkowego wskaźnika per węzeł.

Zjawisko amortyzacji: W przypadku tablicy dynamicznej proces realokacji i alokacji większego bloku pamięci skutkuje sporadycznym skokiem czasu trwania operacji. Mechanizm podwajania pojemności skutecznie "amortyzuje" ten koszt, sprawiając, że dodawanie setek tysięcy rekordów na koniec listy staje się wyjątkowo szybkie.